INSIDE STLC ACADEMY

90-DAY SOFTWARE TESTING CAREER ROADMAP

Regression Testing Checklist

Protect existing features after every change with a risk-based regression pass.

Introduction

Regression testing confirms that recent changes have not broken existing functionality. As a product grows you cannot re-test everything every time, so regression must be risk-based and focused on what the change could affect.

Instructions

- Identify what changed and which areas it could impact.
- Prioritise regression by risk and business impact.
- Always include the core critical paths (login, checkout, key journeys).
- Record results and raise any new defects.

Real QA Example

A change to the discount engine could affect the cart, checkout totals, and tax. The regression pass therefore focuses there first, plus the always-run critical paths, rather than re-testing the whole application.

Regression Checklist

- Core critical paths verified (login, checkout, primary journeys)
- Areas directly impacted by the change tested
- Adjacent/integrated areas checked
- Previously failed/fixed defects re-verified
- Cross-browser / device checks where relevant
- Key API endpoints validated
- Results recorded; new defects raised
- Sign-off against exit criteria

Best Practice Tips

- Automate stable, high-value regression checks so they run every release.

- Use change-impact analysis to scope the pass.
- Keep a living core-regression suite for the critical journeys.

Common Mistakes

- Re-testing everything (slow) or testing only the change (risky).
- Skipping regression under deadline pressure.
- Not re-verifying previously fixed defects.

> **INSIDE STLC PRO TIP**
>
> Risk-based regression beats "run it all". Being able to explain what you focused on and why is exactly the judgement employers hire for.